

Programação Lógica

Lógica de Primeira Ordem

Prof. Edson Alves

Campus UnB Gama: Faculdade de Ciências e Tecnologias em Engenharia

Variáveis e constantes

Definição

Uma **variável** é um símbolo que representa um objeto não especificado de um determinado conjunto U , o qual é chamado **conjunto universal** para a variável. Um membro específico do conjunto universal é denominado **constante**.

Termos em Prolog

Termos em Prolog

- ★ Prolog tem um único tipo de dado, denominado **termo**

Termos em Prolog

- ★ Prolog tem um único tipo de dado, denominado **termo**
- ★ Os argumentos dos predicados Prolog são termos

Termos em Prolog

- ★ Prolog tem um único tipo de dado, denominado **termo**
- ★ Os argumentos dos predicados Prolog são termos
- ★ Um **átomo** é um nome de propósito geral, composto por uma sequência de caracteres, iniciada com letra minúscula

Termos em Prolog

- ★ Prolog tem um único tipo de dado, denominado **termo**
- ★ Os argumentos dos predicados Prolog são termos
- ★ Um **átomo** é um nome de propósito geral, composto por uma sequência de caracteres, iniciada com letra minúscula
- ★ Delimitar um átomo por aspas simples permite o uso de iniciais maiúsculas e espaços em branco

Termos em Prolog

- ★ Prolog tem um único tipo de dado, denominado **termo**
- ★ Os argumentos dos predicados Prolog são termos
- ★ Um **átomo** é um nome de propósito geral, composto por uma sequência de caracteres, iniciada com letra minúscula
- ★ Delimitar um átomo por aspas simples permite o uso de iniciais maiúsculas e espaços em branco
- ★ **Números** (inteiros ou flutuantes) são termos primitivos de Prolog

% Exemplo de declaração de fatos usando átomos e números (constantes)

% Valores em 10/03/2026

cotacao(dolar, 5.16).

cotacao(euro, 6.01).

cotacao(iene, 0.03).

% O primeiro argumento é o nome, o segundo a idade

paciente('Carlos', 38).

paciente('Maria', 26).

paciente('Pedro Lucas', 40).

% Átomos podem conter underscore e números

limites(eixo_x, -10, 10).

limites(eixo_y, -1, 5).

limites(area1, 1, 2).

limites(area2, 0, 3).

Sentenças abertas

Definição

Uma **sentença aberta** $S(x)$ em x é uma expressão na qual o símbolo x ocorre uma ou mais vezes e que, caso todas as ocorrências de x sejam substituídas por uma constante v do conjunto universal de x , $S(v)$ se torna uma proposição. Sentenças abertas também são chamadas **predicados** ou **funções proposicionais**.

Variáveis em Prolog

Variáveis em Prolog

- ★ Em Prolog, as **variáveis** são um subtipo de termo

Variáveis em Prolog

- ★ Em Prolog, as **variáveis** são um subtipo de termo
- ★ Variáveis devem iniciar em letra maiúscula ou com o caractere **'_'**

Variáveis em Prolog

- ★ Em Prolog, as **variáveis** são um subtipo de termo
- ★ Variáveis devem iniciar em letra maiúscula ou com o caractere **'_'**
- ★ As variáveis permitem a declaração de sentenças abertas em Prolog, denominadas **regras**

Variáveis em Prolog

- ★ Em Prolog, as **variáveis** são um subtipo de termo
- ★ Variáveis devem iniciar em letra maiúscula ou com o caractere **'_'**
- ★ As variáveis permitem a declaração de sentenças abertas em Prolog, denominadas **regras**
- ★ A sintaxe para a declaração da regra **rule/N** é

`rule(X1, X2, ..., XN) :- body.`

Variáveis em Prolog

- ★ Em Prolog, as **variáveis** são um subtipo de termo
- ★ Variáveis devem iniciar em letra maiúscula ou com o caractere '_'
- ★ As variáveis permitem a declaração de sentenças abertas em Prolog, denominadas **regras**
- ★ A sintaxe para a declaração da regra **rule/N** é

nome ← rule(X1, X2, ..., XN) :- **body**.

Variáveis em Prolog

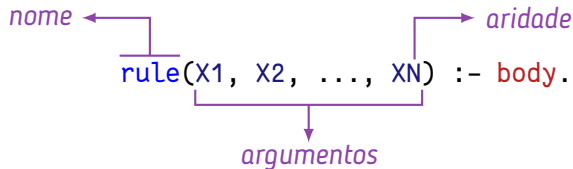
- ★ Em Prolog, as **variáveis** são um subtipo de termo
- ★ Variáveis devem iniciar em letra maiúscula ou com o caractere '_'
- ★ As variáveis permitem a declaração de sentenças abertas em Prolog, denominadas **regras**
- ★ A sintaxe para a declaração da regra **rule/N** é

nome ← rule(X1, X2, ..., XN) :- **body**.

↓
argumentos

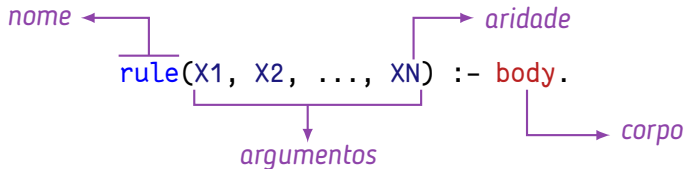
Variáveis em Prolog

- ★ Em Prolog, as **variáveis** são um subtipo de termo
- ★ Variáveis devem iniciar em letra maiúscula ou com o caractere ''
- ★ As variáveis permitem a declaração de sentenças abertas em Prolog, denominadas **regras**
- ★ A sintaxe para a declaração da regra **rule/N** é



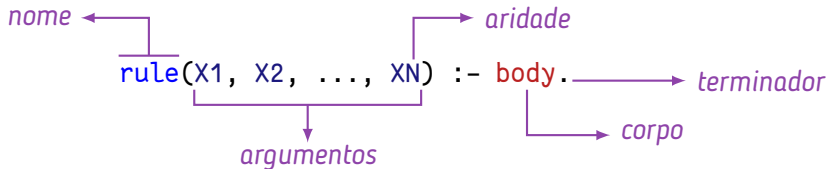
Variáveis em Prolog

- ★ Em Prolog, as **variáveis** são um subtipo de termo
- ★ Variáveis devem iniciar em letra maiúscula ou com o caractere '_'
- ★ As variáveis permitem a declaração de sentenças abertas em Prolog, denominadas **regras**
- ★ A sintaxe para a declaração da regra **rule/N** é



Variáveis em Prolog

- ★ Em Prolog, as **variáveis** são um subtipo de termo
- ★ Variáveis devem iniciar em letra maiúscula ou com o caractere '_'
- ★ As variáveis permitem a declaração de sentenças abertas em Prolog, denominadas **regras**
- ★ A sintaxe para a declaração da regra **rule/N** é



Declarando regras em Prolog

Declarando regras em Prolog

- ★ Assim como os fatos, as regras devem ser declaradas em um arquivo `‘.pl’`

Declarando regras em Prolog

- ★ Assim como os fatos, as regras devem ser declaradas em um arquivo `‘.pl’`
- ★ O corpo de uma regra deve ou ser uma proposição composta ou ser omitido

Declarando regras em Prolog

- ★ Assim como os fatos, as regras devem ser declaradas em um arquivo `‘.pl’`
- ★ O corpo de uma regra deve ou ser uma proposição composta ou ser omitido
- ★ Para diferenciar regras de fatos neste caso, observe que, em uma regra, ao menos um dos argumentos deve ser uma variável:

```
f(1).           % Fato  
f(X).           % Regra, equivale a f(X) :- true.
```


Declarando regras em Prolog

- ★ Assim como os fatos, as regras devem ser declaradas em um arquivo `‘.pl’`
- ★ O corpo de uma regra deve ou ser uma proposição composta ou ser omitido
- ★ Para diferenciar regras de fatos neste caso, observe que, em uma regra, ao menos um dos argumentos deve ser uma variável:

```
f(1).           % Fato  
f(X).           % Regra, equivale a f(X) :- true.
```

- ★ Também como nos fatos, podemos ter múltiplas regras para o mesmo predicado: o predicado será determinado pela disjunção de todas as regras e fatos declarados

Declarando regras em Prolog

- ★ Assim como os fatos, as regras devem ser declaradas em um arquivo `‘.pl’`
- ★ O corpo de uma regra deve ou ser uma proposição composta ou ser omitido
- ★ Para diferenciar regras de fatos neste caso, observe que, em uma regra, ao menos um dos argumentos deve ser uma variável:

```
f(1).           % Fato  
f(X).           % Regra, equivale a f(X) :- true.
```

- ★ Também como nos fatos, podemos ter múltiplas regras para o mesmo predicado: o predicado será determinado pela disjunção de todas as regras e fatos declarados
- ★ Em Prolog, fatos e regras são denominados **cláusulas**

% Exemplo de declaração de regras em Prolog

% Fatos sobre estudantes da FCTE, do campus Darcy Ribeiro e do IFB

fcte(ana).

fcte(beto).

fcte(carlos).

darcy(diana).

darcy(esther).

ifb(fernando).

% Predicado que identifica alunos da UnB

unb(X) :- (fcte(X) ; darcy(X)).

% Implementação alternativa, em duas regras

unb2(X) :- fcte(X).

unb2(X) :- darcy(X).

Quantificador existencial

Definição

Sejam $S(x)$ uma sentença aberta. O quantificador **existencial** $\exists$ é utilizado na construção $\exists x.S(x)$, a qual significa que existe pelo menos um elemento v no conjunto universal de x tal que $S(v)$ é verdadeira.

Consultas simples

Consultas simples

- ★ Em Prolog, quantificador existencial está associado às consultas

Consultas simples

- ★ Em Prolog, quantificador existencial está associado às consultas
- ★ Uma **consulta simples** corresponde a uma sentença aberta simples, isto é, um predicado e seus respectivos argumentos

Consultas simples

- ★ Em Prolog, quantificador existencial está associado às consultas
- ★ Uma **consulta simples** corresponde a uma sentença aberta simples, isto é, um predicado e seus respectivos argumentos
- ★ Se um ou mais argumentos da consulta forem substituídos por uma variável lógica e o resultado desta consulta for verdadeiro, o interpretador indicará as atribuições que tornaram a sentença verdadeira

Consultas simples

- ★ Em Prolog, quantificador existencial está associado às consultas
- ★ Uma **consulta simples** corresponde a uma sentença aberta simples, isto é, um predicado e seus respectivos argumentos
- ★ Se um ou mais argumentos da consulta forem substituídos por uma variável lógica e o resultado desta consulta for verdadeiro, o interpretador indicará as atribuições que tornaram a sentença verdadeira
- ★ Caso exista duas ou mais atribuições possíveis que tornem a consulta verdadeira, elas podem ser obtidas digitando o caractere ';' no interpretador

% Resultados de consultas simples para as cláusulas sobre estudantes

?- unb(fernando).

false.

?- unb(diana).

true.

% X é uma variável lógica (inicia em maiúscula)

?- ifb(X).

X = fernando.

% Observe o uso de ponto-e-vírgula para as demais atribuições

?- unb(X).

X = ana ;

X = beto ;

X = carlos ;

X = diana ;

X = esther.

Consultas e variáveis lógicas

Consultas e variáveis lógicas

★ As consultas em Prolog são avaliadas por meio de um mecanismo de casamento de padrões denominado **unificação**

Consultas e variáveis lógicas

- ★ As consultas em Prolog são avaliadas por meio de um mecanismo de casamento de padrões denominado **unificação**
- ★ O padrão de uma consulta (predicados e argumentos) é denominado **objetivo** (*goal*)

Consultas e variáveis lógicas

- ★ As consultas em Prolog são avaliadas por meio de um mecanismo de casamento de padrões denominado **unificação**
- ★ O padrão de uma consulta (predicados e argumentos) é denominado **objetivo** (*goal*)
- ★ As variáveis lógicas não se comportam como as variáveis das outras linguagens

Consultas e variáveis lógicas

- ★ As consultas em Prolog são avaliadas por meio de um mecanismo de casamento de padrões denominado **unificação**
- ★ O padrão de uma consulta (predicados e argumentos) é denominado **objetivo** (*goal*)
- ★ As variáveis lógicas não se comportam como as variáveis das outras linguagens
- ★ Durante a unificação, uma variável lógica **casa** com qualquer termo

Consultas e variáveis lógicas

- ★ As consultas em Prolog são avaliadas por meio de um mecanismo de casamento de padrões denominado **unificação**
- ★ O padrão de uma consulta (predicados e argumentos) é denominado **objetivo** (*goal*)
- ★ As variáveis lógicas não se comportam como as variáveis das outras linguagens
- ★ Durante a unificação, uma variável lógica **casa** com qualquer termo
- ★ Após a unificação, uma variável lógica terá como valor o termo que casou (atou, *to bind*) com ela durante a unificação

Unificação

Versão simplificada

Se a base Prolog contém apenas fatos, a unificação é bem sucedida se as três condições abaixo são satisfeitas:

1. o predicado presente no objetivo e na base de dados é o mesmo
2. ambos tem mesma aridade
3. todos os argumentos são os mesmos

% Os fatos abaixo devem ser carregados no interpretador Prolog

f(1, 2).

f(1, 3).

f(2, 3).

g(1).

g(2).

% Exemplos de consultas (unificação simplificada)

?- f(1, 2).

true.

?- f(2, 1).

false.

?- f(1).

false.

?- g(1, 2).

false.

% Fatos a serem declarados em um arquivo '.pl'

algoritmo(**grafos**, **kruskall**).

algoritmo(**grafos**, **bellman-ford**).

algoritmo(**strings**, **kmp**).

algoritmo(**strings**, **z-function**).

algoritmo(**strings**, **lcs**).

algoritmo(**gulosos**, **kruskall**).

% Exemplos de consultas (unificação simples + variáveis)

?- algoritmo(**strings**, X).

X = **kmp** ;

X = **z-function** ;

X = **lcs**.

?- algoritmo(**Area**, **kruskall**).

Area = **grafos** ;

Area = **gulosos**.

% Mais uma exemplo de unificação simples

f(1, 1).

f(1, 2).

f(1, 3).

f(2, 1).

f(2, 2).

f(2, 3).

g(a, 1).

g(1, a).

% Exemplos de consultas com mesma variável lógica em ambos argumentos

?- f(X, X).

X = 1 ;

X = 2 ;

false.

?- g(X, X).

false.

Fatos dinâmicos

Fatos dinâmicos

- ★ Prolog permite a declaração ou retratação de fatos diretamente no interpretador

Fatos dinâmicos

- ★ Prolog permite a declaração ou retratação de fatos diretamente no interpretador
- ★ Para isto, o predicado deve ser declarado **dinâmico**, por meio da diretiva `dynamic`

Fatos dinâmicos

- ★ Prolog permite a declaração ou retratação de fatos diretamente no interpretador
- ★ Para isto, o predicado deve ser declarado **dinâmico**, por meio da diretiva `dynamic`
- ★ Deve ser indicado o nome e a aridade do predicado, e essa declaração só pode ser feita em arquivo

Fatos dinâmicos

- ★ Prolog permite a declaração ou retratação de fatos diretamente no interpretador
- ★ Para isto, o predicado deve ser declarado **dinâmico**, por meio da diretiva `dynamic`
- ★ Deve ser indicado o nome e a aridade do predicado, e essa declaração só pode ser feita em arquivo
- ★ Os predicados `asserta/1` e `assertz/1` adicionam um novo fato ao predicado dinâmico no início ou no fim da base Prolog, respectivamente

Fatos dinâmicos

- ★ Prolog permite a declaração ou retratação de fatos diretamente no interpretador
- ★ Para isto, o predicado deve ser declarado **dinâmico**, por meio da diretiva `dynamic`
- ★ Deve ser indicado o nome e a aridade do predicado, e essa declaração só pode ser feita em arquivo
- ★ Os predicados `asserta/1` e `assertz/1` adicionam um novo fato ao predicado dinâmico no início ou no fim da base Prolog, respectivamente
- ★ O predicado `retract/1` apaga um fato da base Prolog

```
:- dynamic  
    coordenador/1.
```

```
coordenador(matheus).      % CNA  
coordenador(himilsys).     % Automotiva  
coordenador(batistuta).    % Eletrônica  
coordenador(sebastien).    % Aeroespacial
```

% Após carregar estes fatos, são possíveis as seguintes consultas/atualizações

% Lista os coordenadores

```
?- coordenador(X).
```

```
X = matheus ;  
X = himilsys ;  
X = batistuta ;  
X = sebastien.
```

% Verifica se John Lenon é coordenador de Software

```
?- coordenador(john).  
false.
```

% Atualiza, adicionando os coordenadores ainda não cadastrados

*?- asserta(coordenador(**andrea**)).* *% Energia*

true.

*?- assertz(coordenador(**john**)).* *% Software*

true.

% Agora John é coordenador

*?- coordenador(**john**).*

true.

% Suponha que Matheus se licencie do cargo de coordenador

*?- retract(coordenador(**matheus**)).*

true.

?- coordenador(X).

X = andrea ;

X = himilsys ;

X = batistuta ;

X = sebastien ;

X = john.

Quantificador universal

Definição

Sejam $S(x)$ uma sentença aberta. O quantificador **universal** $\forall$ é utilizado na construção $\forall x.S(x)$, a qual significa que, para todos os valores v no conjunto universal de x , $S(v)$ é verdadeira.

Generalizações

Generalizações

- ★ O quantificador universal é expresso, em Prolog, por meio de uma **generalização**

Generalizações

- ★ O quantificador universal é expresso, em Prolog, por meio de uma **generalização**
- ★ Generalizações são regras que casam com qualquer valor

Generalizações

- ★ O quantificador universal é expresso, em Prolog, por meio de uma **generalização**
- ★ Generalizações são regras que casam com qualquer valor
- ★ Por exemplo, considere o predicado `is_term/1`, definido abaixo:

```
is_term(X).
```

Generalizações

- ★ O quantificador universal é expresso, em Prolog, por meio de uma **generalização**
- ★ Generalizações são regras que casam com qualquer valor
- ★ Por exemplo, considere o predicado `is_term/1`, definido abaixo:

```
is_term(X).
```

- ★ Ele é verdadeiro para qualquer argumento X: de fato, o argumento de um predicado deve ser um termo, de modo que a implementação está semanticamente correta

Generalizações

- ★ O quantificador universal é expresso, em Prolog, por meio de uma **generalização**
- ★ Generalizações são regras que casam com qualquer valor
- ★ Por exemplo, considere o predicado `is_term/1`, definido abaixo:

```
is_term(X).
```

- ★ Ele é verdadeiro para qualquer argumento X: de fato, o argumento de um predicado deve ser um termo, de modo que a implementação está semanticamente correta
- ★ Uma vez que a variável X não é utilizada no corpo da regra, ela pode ser substituída pelo *wildcard* `'_'`

% Definição da generalização usando o wildcard

`is_term(_).`

% Todas as consultas a seguir são verdadeiras

`?- is_term(1).` *% Números são termos*

`true.`

`?- is_term(abc).` *% Átomos são termos*

`true.`

`?- is_term(A).` *% Variáveis são termos*

`true.`

% A consulta a seguir gera um erro, pois is_term tem aridade 1

`?- is_term(A, B).`

Referências

- ★ **Banco Central do Brasil.** *Cotações e boletins*, acesso em 10/03/2026.
- ★ **LibreTexts Mathematics.** *Open Sentences and Sets*, acesso em 10/03/2026.
- ★ **MERRIT, Dennis.** *Adventure in Prolog, Amzi!*, 191 pgs, 2017.
- ★ **SWI-Prolog.** <https://www.swi-prolog.org/>, acesso em 10/03/2026.
- ★ **Wikipédia.** *Prolog syntax and semantics*, acesso em 10/03/2026.